

Yes! You can run LLMs on Kubernetes

Abdel Sghiouar



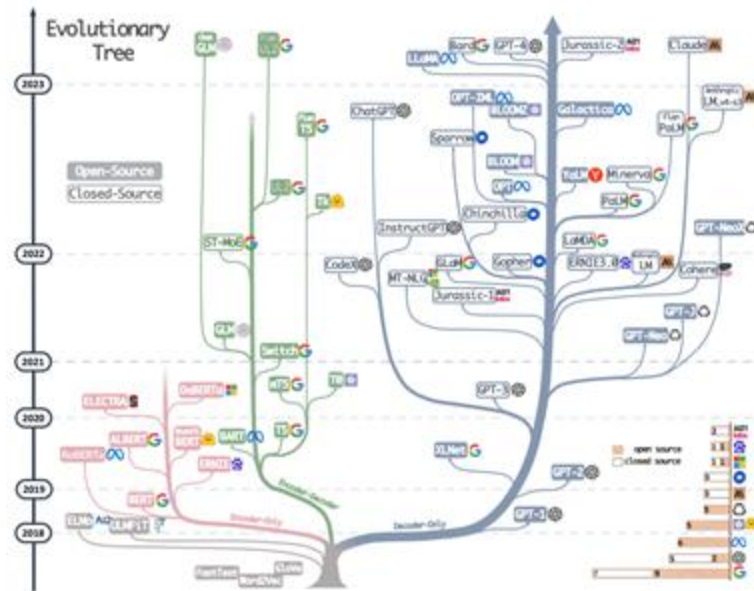
Abdel Sghiouar
@boredabdel.bsky.social

Thank you



Generative AI models are evolving rapidly

- GenAI capabilities, reasoning and performance are continually improving
- Open foundation models are enabling rapid innovation in the AI community
 - Flexible deployment
 - Task and domain specific fine-tuning
 - Resource optimization



<https://arxiv.org/pdf/2304.13712v2.pdf>

Open LLMs

DeepSeek R1

Open Weights Reasoning Model
671B

Llama 3

1B, 3B, 11B, 70B, 90B, 405B

Gemma 3

1B, 4B, 12B, 27B

Qwen

FLAN-T5

SDXL

Falcon 7B/40B

Phi

Mistral

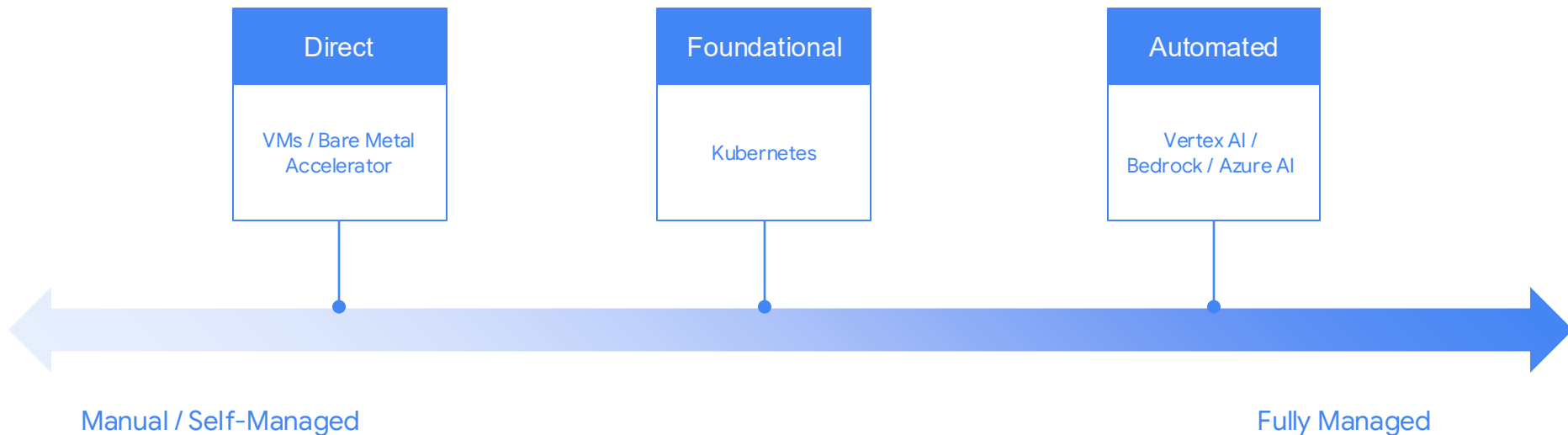
...many more

LLM is the new Web App*

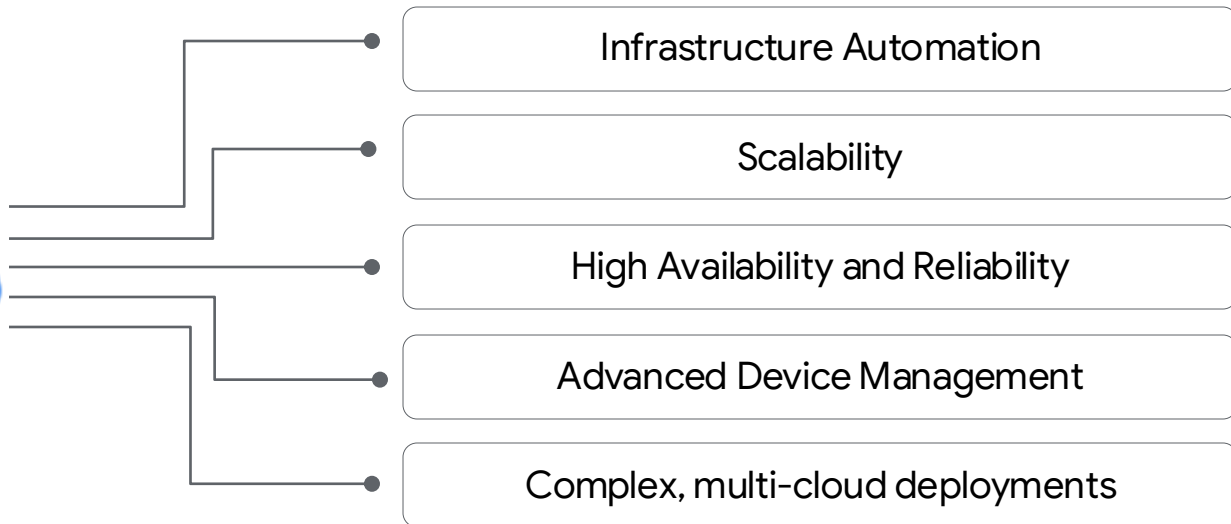
Clayton Coleman



Options for running LLMs in Cloud



K8s is great for running web apps



K8s is great for running web apps



Infrastructure **Automation**

Scalability

High Availability and Reliability

Advanced Device Management

Complex, **multi-cloud deployments**

Demo



LLM is not a typical Web App*



Concepts

Serving Engine

Standardize Serving of LLM and gives you an API server.

Weights

The numerical values that defines the strength between neurons

LLM

Machine Learning Models trained on vast amounts of data that can comprehend* and generate human language text

Accelerators

Purpose built hardware for matrix calculation (GPU, TPU, GROQ)

Context Window

How many token (words) a model can ingest/output in one request

Inference/serving

Making a model available for end users/apps via an API

Quantization

Changing the model precision to reduce memory footprint

Multi-Modal

Text, Audio, Pictures, Videos...

Commonly used model servers



Hugging Face
Text Generation
Inference (TGI)



Ray-Serve



Jetstream



Ollama

Typical Inference server image sizes

Image	Image Size	Avg Image Pull Time
pytorch/pytorch:2.5.1-cuda12.4-cudnn9-devel	6.91 GB	2m25.82
vllm/vllm-openai:v0.6.4.post1	5.24 GB	1m55.701s
pytorch/torchserve:0.12.0-cpp-dev-gpu	4.8 GB	1m36.114
redhat/granite-3-8b-instruct:1.0-1731549069	5.21 GB	2m3.14s
paddlepaddle/paddle:2.6.2-gpu-cuda11.7-cudnn8.4-trt8.4	9.62 GB	2m41.304s
paddlepaddle/paddle:3.0.0b1-gpu-cuda11.8-cudnn8.6-trt8.5	10.08 GB	1m51.797s
paddlepaddle/paddle:3.0.0b1-gpu-cuda11.8-cudnn8.9-trt8.5	8.7 GB	2m29.802s
paddlepaddle/paddle:2.6.0-gpu-cuda12.0-cudnn8.9-trt8.6	14.56 GB	5m31.974s

Model Size

Models	Model Variants	Model Size (GPU/TPU memory needed in GB)		
		FP16	8-bit Quantization	4-bit Quantization
Gemma	1b	2	1	.5
	4b	8	4	2
	27b	54	27	13.5
Llama 3	3b	14	7	3.5
	11b	22	11	5.5
	70b	140	70	35
Falcon	7b	14	14	3.5
	40b	80	40	20
	180b	360	180	90
Flan T5	11b	22	11	5.5
Deepseek	685b	1370	685	392

*model size (in bytes) = model parameters * data type in bytes*

7b model in 16-bit precision:

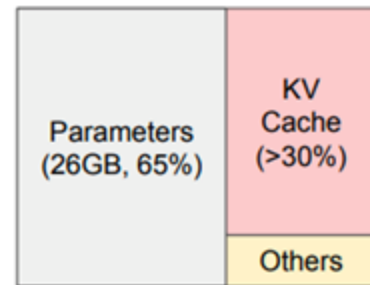
*7 billion * 2 = 14 billion bytes = 14 GiB*

7b model in 8-bit precision:

*7 billion * 1 = 7 billion bytes = 7 GiB*

7b model in 4-bit precision:

*7 billion * 0.5 = 3.5 billion bytes = 3.5 GiB*



NVIDIA A100 40GB

Concepts Cont.

**Single Host Single
Accelerator**

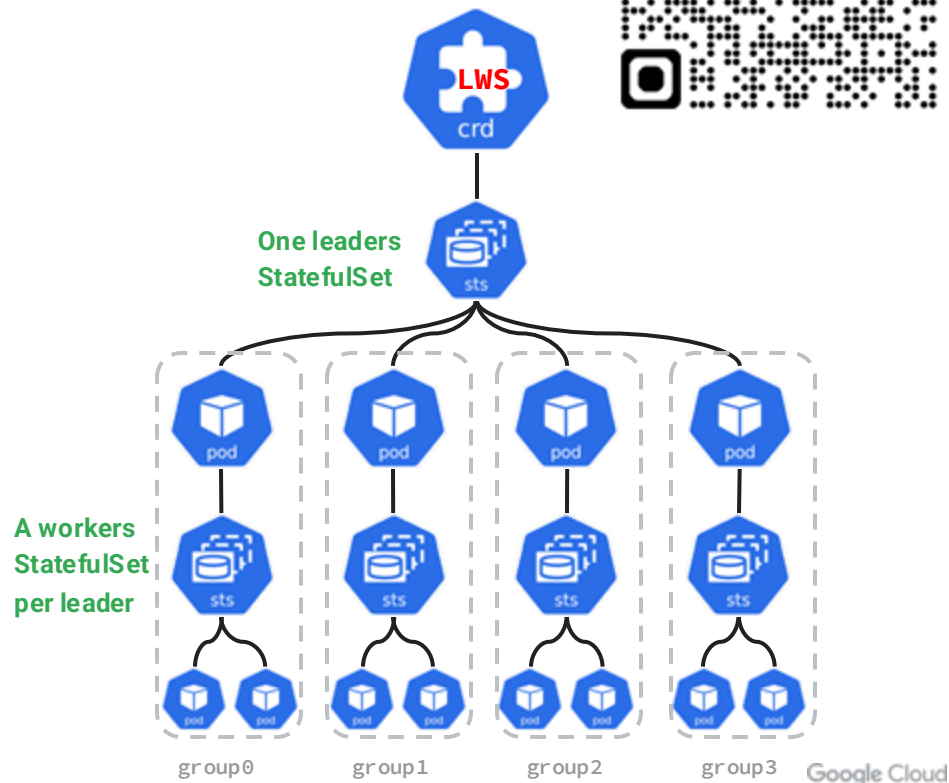
**Single Host Multi
Accelerator**

**Multi Host Multi
Accelerators**

Inference Optimized TPU-stack allows 1 trillion parameter model deployment



- Seamless run large models such Llama 405B and Deepseek R1 671B on TPUs thanks to best-in-class parallelization from Pathways technology
- Kubernetes orchestrates multi-node deployments using Leader Worker Set, a component contributed to Kubernetes by Google and has now become the standard for multi-host inference orchestration



Layers and dimension of LLM Optimization

Model Optimization

Tune or quantize the model to deliver the acceptable performance at the lowest unit operating cost

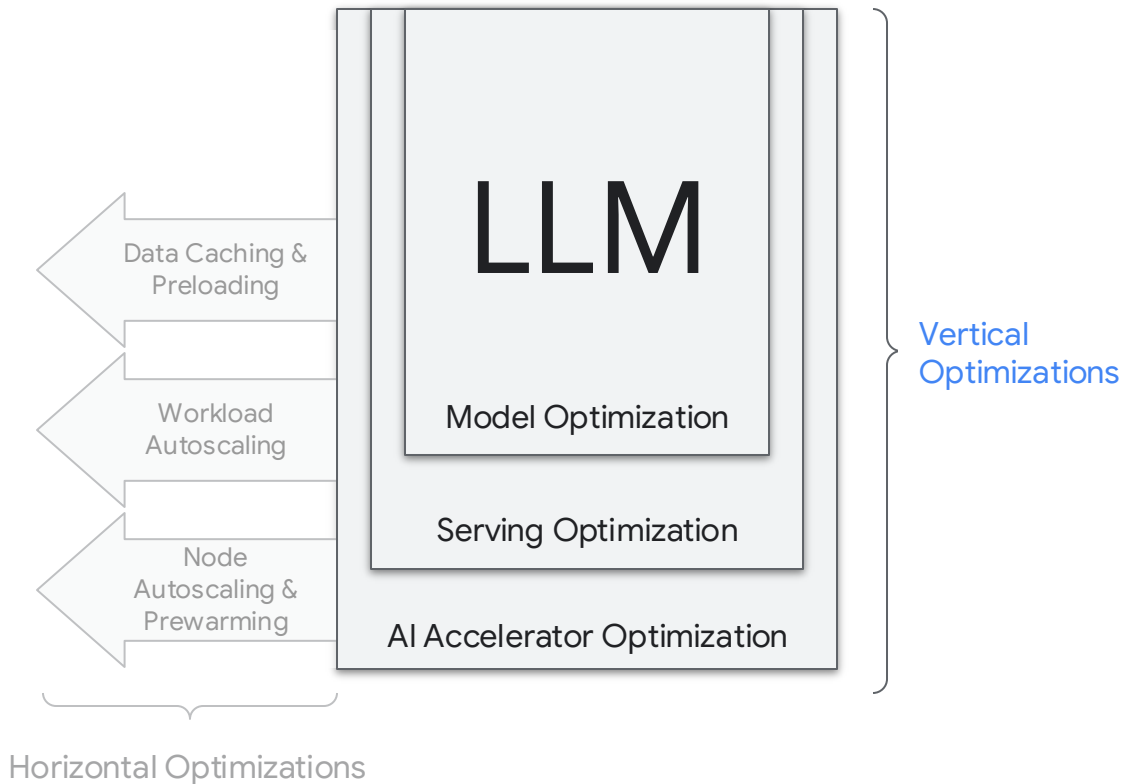
Serving Optimization

Leverage technics and serving stacks to maximize throughput and minimize latency

AI Accelerator Optimization

Select and configure GPUs and TPUs to maximize utilization

Vertical Optimization will be the primary focus of today's session





<https://github.com/moficodes/multi-llm>

Blazing-Fast Scheduling Performance

General Purpose Workloads

Up to 7× Faster Pod Scheduling

For standard, autoscaled applications running on the Autopilot compute class, pods launch faster. This is perfect for stateless apps, web services that gradually autoscale, where speed and rapid scaling are critical.

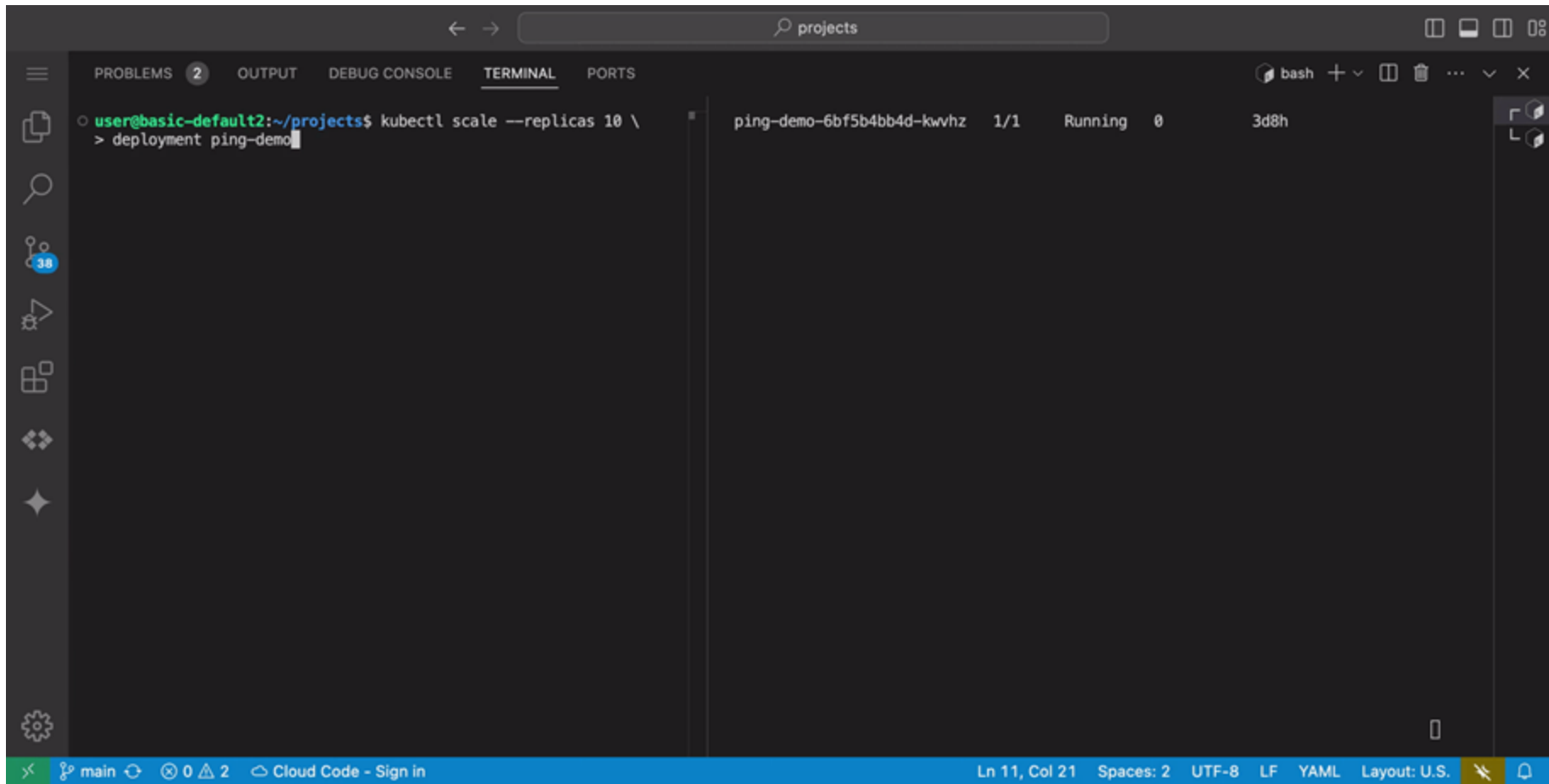
- **Benefit:** Respond to traffic spikes in real-time and cut wait times and overprovisioning.

Accelerator Workloads

Up to 2× Faster Pod Scheduling

For demanding AI and Machine Learning tasks, we've accelerated pod startup on specialized hardware starting with L4 GPUs on the G2 machine family (limitations apply)

Container Optimized Compute



The screenshot shows a VS Code editor interface with a terminal window open. The terminal is running a bash shell. The command entered is `kubectl scale --replicas 10 \> deployment ping-demo`. The output shows the status of the deployment: `ping-demo-6bf5b4bb4d-kwvhz 1/1 Running 0 3d8h`.

VS Code interface details:

- Search bar at the top: `projects`
- Terminal tabs: PROBLEMS (2), OUTPUT, DEBUG CONSOLE, **TERMINAL**, PORTS
- Terminal title bar: `bash` with window controls
- Terminal content:

```
user@basic-default2:~/projects$ kubectl scale --replicas 10 \
> deployment ping-demo
```

```
ping-demo-6bf5b4bb4d-kwvhz 1/1 Running 0 3d8h
```
- Bottom status bar: `main`, `0 2`, `Cloud Code - Sign in`, `Ln 11, Col 21`, `Spaces: 2`, `UTF-8`, `LF`, `YAML`, `Layout: U.S.`

Container Optimized Compute

Up to 7x* faster pod scheduling

50% less idle capacity required to handle **3.5x increase** in traffic

Low latency autoscaling reaction time

~35% less replicas required to meet latency requirements for scaleup

Non-disruptive vertical scalability

With **in-place pod** updates, **non-disruptive resizing** becomes reality

Available out of the box

Now available as default compute for Autopilot on Rapid channel!

7X

faster scheduling

In-Place Pod Resize in GKE 1.33 Preview

1. Update kubectl to 1.33 or newer
2. Create a cluster with version **1.33.0-gke.1552000** or newer
3. Create a Pod
4. Resize the pod using 'patch':

```
kubectl patch pod <pod-name> --subresource resize --patch \
'{"spec":{"containers":[{"name":"<container-name>",
"resources":{"requests":{"cpu":"100"}, "limits":{"cpu":"100"}}}]}}'
```

Official documentation: <https://kubernetes.io/docs/tasks/configure-pod-container/resize-container-resources/>

Fast-Starting GKE Nodes

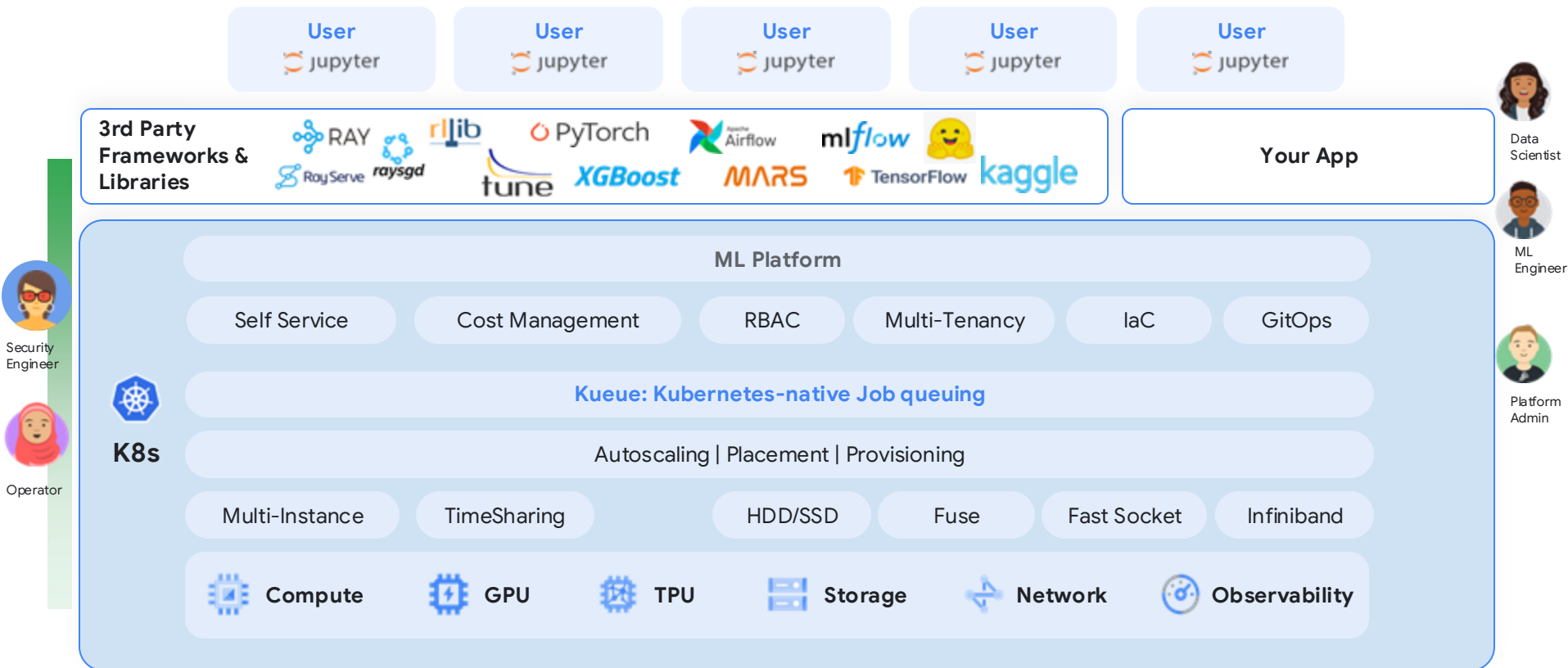
Accelerate Workload Provisioning

Now Under a Minute to initialize a new node with an Nvidia L4 accelerator

- Dramatically Reducing Cold Startup Times: Improving GKE Autopilot node cold startup time by up to 90% and achieving blazing fast cold startup times (e.g., G2-L4 nodes are now 4x faster at 45 seconds, p95).
- Accelerating Scale-Ups: Providing noticeably faster scale-ups for Vertex AI users, enabling more agile and responsive operations.
- Enhancing Competitiveness: Closing competitive gaps by making Vertex AI node cold start performance equivalent to or faster than AWS for the first time, ensuring a leading-edge experience.
- Effortless Benefit: Delivering these performance improvements with absolutely no configuration required from the customer – the enhancements are entirely "under the hood."

```
apiVersion: cloud.google.com/v1
kind: ComputeClass
metadata:
  name: my-fast-starting-accelerator-compute-class
  // You may optionally validate that the first
  // priority is fast-start compatible.
  annotations:
    gke.io/fast-node-startup: validate-first-
    priority
spec:
  autopilot:
    enabled: true
  priorities:
    - machineFamily: g2
      gpu:
        count: 1
  nodePoolAutoCreation:
    enabled: true
```


Reference Architecture for AI/ML Platforms



\$ Gracias

→ Abdel Sghiouar

- Senior Cloud Developer Advocate @Google
- Co-Host of the Kubernetes Podcast from Google
- Kubernetes, Containers, Service Mesh and Security
- Moroccan Living in Sweden

